

Direct Digitzs/NMI Integration (Backup Solution)

Overview

Direct integration to Digitzs (NMI white label) using the same Propay MID 33595002 that was configured through Tokenex.

Why Direct Integration?

Pros

- No dependency on Tokenex
- Lower monthly costs (no Tokenex fees)
- Faster transaction processing (one less hop)
- Full control over payment flow

Cons

- Increased PCI compliance scope
- Card data touches your frontend
- Limited to NMI/Digitzs processors only
- Must implement own tokenization

Architecture

Payment Flow

Customer → Your Hosted Form → Your Edge Function → Digitzs API → Propay MID 33595002

Security Considerations

1. Use HTTPS everywhere
2. Never log card numbers
3. Use Digitzs's Customer Vault for tokenization

4. Implement proper CSP headers
5. Add rate limiting to prevent abuse

API Integration

Endpoint

```
POST https://digitzs.transactiongateway.com/api/transact.php
```

Authentication

Uses security key in POST body (not headers)

Request Format (Sale Transaction)

```
POST /api/transact.php HTTP/1.1
Host: digitzs.transactiongateway.com
Content-Type: application/x-www-form-urlencoded

security_key=YOUR_SECURITY_KEY
&type=sale
&amount=45.00
&ccnumber=4111111111111111
&ccexp=1225
&cvv=999
&firstname=John
&lastname=Doe
&email=john@example.com
&phone=5555555555
&address1=123 Main St
&city=Austin
&state=TX
&zip=78701
&country=US
&orderid=ORDER-123
&ipaddress=192.168.1.1
```

Response Format (Success)

```
response=1
&responsetext=SUCCESS
&authcode=123456
&transactionid=7890123456
&avsresponse=Y
&cvvresponse=M
&orderid=ORDER-123
&response_code=100
```

Response Codes

- **1** = Approved
- **2** = Declined
- **3** = Error

Customer Vault (Tokenization)

Why Use Customer Vault?

- Store card data securely at NMI/Digitzs
- Reduces your PCI scope
- Enable recurring payments
- Fast repeat purchases

Add Customer

```
POST /api/transact.php
security_key=YOUR_SECURITY_KEY
&customer_vault=add_customer
&ccnumber=4111111111111111
&ccexp=1225
&firstname=John
&lastname=Doe
&email=john@example.com
```

Response

```
response=1  
&customer_vault_id=1234567890
```

Charge Using Vault

```
POST /api/transact.php  
security_key=YOUR_SECURITY_KEY  
&type=sale  
&amount=45.00  
&customer_vault_id=1234567890  
&cvv=999
```

Implementation

Edge Function Structure

```
// supabase/functions/digitzs-process/index.ts

interface DigitzsRequest {
  amount: number;
  cardNumber: string;
  cardExpiry: string; // MMY format
  cvv: string;
  customerInfo: {
    firstName: string;
    lastName: string;
    email: string;
    phone: string;
    address: string;
    city: string;
    state: string;
    zip: string;
  };
  orderId: string;
  ipAddress: string;
}

// Convert to form-encoded format
const formData = new URLSearchParams({
  security_key: DIGITZS_SECURITY_KEY,
  type: 'sale',
  amount: amount.toFixed(2),
  ccnumber: cardNumber,
  ccexp: cardExpiry,
  cvv: cvv,
  firstname: firstName,
  lastname: lastName,
  email: email,
  // ... more fields
});

// POST to Digitzs
const response = await fetch(
  'https://digitzs.transactiongateway.com/api/transact.php',
  {
    method: 'POST',
    headers: {
      'Content-Type': 'application/x-www-form-urlencoded',
    },
    body: formData.toString(),
  }
);
```

Frontend Form

```
// Secure payment form component
<form onSubmit={handleSubmit}>
  <input
    type="text"
    maxLength={19}
    placeholder="Card Number"
    autoComplete="cc-number"
  />
  <input
    type="text"
    maxLength={4}
    placeholder="MMYY"
    autoComplete="cc-exp"
  />
  <input
    type="text"
    maxLength={4}
    placeholder="CVV"
    autoComplete="cc-csc"
  />
  { /* Customer info fields */ }
</form>
```

Testing

Test Cards (Digitzs/NMI)

```
Visa Approval: 4111111111111111
Visa Decline: 4000300011112220
Mastercard Approval: 5499740000000057
Amex Approval: 371449635398431

CVV: Any 3-4 digits
Expiry: Any future date (MMYY format)
```

Test Mode

- Use test security key from Digitzs portal
- Test transactions don't hit real card networks
- Test data doesn't appear in production reports

Error Handling

Common Errors

```
const errorMessages: Record<string, string> = {
  '200': 'Transaction declined',
  '201': 'Do not honor',
  '202': 'Insufficient funds',
  '203': 'Over limit',
  '204': 'Transaction not allowed',
  '220': 'Incorrect payment information',
  '221': 'No such card issuer',
  '222': 'No card number on file',
  '223': 'Expired card',
  '224': 'Invalid expiration date',
  '225': 'Invalid security code',
  '240': 'Call issuer for further information',
  '250': 'Pick up card',
  '251': 'Lost card',
  '252': 'Stolen card',
  '253': 'Fraudulent card',
  '260': 'Declined with further instructions',
  '261': 'Declined - stop all recurring payments',
  '262': 'Declined - stop this recurring program',
  '263': 'Declined - update cardholder data available',
  '264': 'Declined - retry in a few days',
  '300': 'Transaction rejected',
  '400': 'Transaction error returned by processor',
  '410': 'Invalid merchant configuration',
  '411': 'Merchant account is inactive',
  '420': 'Communication error',
  '421': 'Communication error with issuer',
  '430': 'Duplicate transaction',
  '440': 'Processor format error',
  '441': 'Invalid transaction information',
  '460': 'Processor feature not available',
  '461': '3D Secure required',
};
```

Security Best Practices

1. Environment Variables

```
# Store in Supabase Edge Function Secrets
DIGITZS_SECURITY_KEY=your_security_key_here
DIGITZS_MERCHANT_ID=33595002
```

2. Input Validation

```
// Validate card number (Luhn algorithm)
function isValidCardNumber(cardNumber: string): boolean {
  // Implementation
}

// Validate expiry
function isValidExpiry(expiry: string): boolean {
  const [month, year] = expiry.match(/../g) || [];
  // Check future date
}

// Validate CVV
function isValidCVV(cvv: string, cardType: string): boolean {
  return /^\\d{3,4}$/.test(cvv);
}
```

3. Rate Limiting

```
// Implement rate limiting to prevent abuse
const rateLimitKey = `ratelimit:${ipAddress}`;
const attempts = await redis.incr(rateLimitKey);
if (attempts > 5) {
  throw new Error('Too many attempts');
}
await redis.expire(rateLimitKey, 3600); // 1 hour
```

4. Logging

```
// NEVER log card numbers or CVV
console.log({
  orderId,
  amount,
  lastFour: cardNumber.slice(-4),
  // NEVER log full card number
  result: response.responsetext,
  transactionId: response.transactionid,
});
```


Migration from Tokenex

Data Mapping

Tokenex Field	Digitzs Field	Notes
Token	customer_vault_id	Use Customer Vault
Amount	amount	Same format
MerchantID	Built into security_key	No separate field
Gateway	N/A	Direct to Digitzs
TransactionType	type	sale/auth/capture/void

Code Changes Required

1. Remove Tokenex iFrame component
2. Add card input form
3. Update edge function to call Digitzs directly
4. Implement Customer Vault for tokenization
5. Update error handling for Digitzs response codes
6. Add additional PCI compliance measures

PCI Compliance Requirements

SAQ A-EP (Self-Assessment Questionnaire)

When using direct integration without iFrame:

- Use HTTPS everywhere
- Don't store CVV ever
- Tokenize cards via Customer Vault
- Log access to cardholder data
- Use strong cryptography
- Maintain secure systems
- Implement access controls

- Monitor and test networks regularly

Reducing PCI Scope

1. **Use Customer Vault:** Store cards at Digitzs
2. **Don't store CVV:** Never save CVV anywhere
3. **Use TLS 1.2+:** Encrypt data in transit
4. **Minimize card data:** Only collect what's needed
5. **Use tokens:** Reference vault IDs instead of cards

Production Checklist

- ☐ Switch to production security key
- ☐ Test with real cards in small amounts
- ☐ Implement error logging
- ☐ Set up transaction monitoring
- ☐ Configure fraud detection settings in Digitzs portal
- ☐ Test refund/void functionality
- ☐ Verify TicketSocket integration
- ☐ Set up automated reconciliation
- ☐ Document support procedures
- ☐ Train staff on new flow

Rollback Plan

If issues arise with direct integration:

1. Keep Tokenex code in separate branch
2. Document all Tokenex credentials
3. Maintain Tokenex account relationship
4. Test Tokenex reactivation in sandbox first
5. Have feature flag to switch between integrations

Cost Comparison

Tokenex + Digitzs

- Tokenex: ~\$500-1000/month base + per-transaction
- Processor fees: 2.9% + \$0.30

Direct Digitzs

- No Tokenex fees
- Processor fees: 2.9% + \$0.30
- Potential savings: \$6,000-12,000/year

Support

Digitzs Support

- Email: support@nmi.com
- Phone: (800) 617-4850
- Portal: <https://secure.digitzs.transactiongateway.com>
- Docs:
https://secure.digitzs.transactiongateway.com/merchants/resources/integration/integration_portal.php

Emergency Contacts

- MID: 33595002
- Security Key: (stored in secrets)
- Account Rep: (add contact info)